



Universitat Autònoma de Barcelona
Enginyeria Informàtica
Curso 2026-2027
Sistemas Operativos
Práctica 1. Procesos



Profesores de prácticas: Edixon Parraga, Vicente Ivars, Marc Violán y Ramon González

Material de las prácticas

Para esta práctica no es necesaria más documentación que la que se describe en este enunciado. No obstante, os proporcionamos una lista de referencias escogidas para buscar información adicional para iniciarse y para entrar más en profundidad.

Referencias:

- Sistemas Operativos. Pedro de Miguel Anasagasti, Fernando Pérez Costoya. UPM. 2016
- Advance Programming in the UNIX Environment. Stevens Rago. Addison-Wesley
- The Linux Documentation Project: <https://tldp.org/LDP/tlk/kernel/processes.html>

Objetivos y habilidades

- Conocer las llamadas al sistema para gestionar procesos y tuberías.
- Conocer los mecanismos básicos de sincronización entre procesos (IPC).
- Desarrollar programas concurrentes con procesos.

Entorno y entrega

Tomaremos como referencia la versión Linux Debian 64 bits del laboratorio de prácticas, aunque la mayoría de ejercicios funcionan igual en otras distribuciones como Ubuntu 18.04 o 20.04. Deberéis contestar las respuestas a las preguntas realizadas en este enunciado de forma ordenada justificando cada respuesta en un documento con el nombre de los componentes del grupo y entregarlo antes de que se termine el plazo de entrega de la práctica en el campus virtual.

Ejercicio 1: Creación de procesos

En este primer ejemplo ('pexample1.c') aprenderemos como crear procesos y esperar a que finalicen. En el siguiente código podemos ver como el padre crea un proceso hijo utilizando la llamada al sistema a fork(). Se imprime por pantalla su timestamp y espera a que el hijo termine su ejecución llamando a wait(). Por su lado, el hijo también imprime su timestamp y termina su ejecución llamando a exit().

```
int main(int argc, char** argv) {
    int status;
    int pid;
    // Create a process (fork)
    if ((pid=fork())==0) {
        // Child process
        printf("[%s] Child process (PID=%d)\n", timestamp(), getpid());
        exit(0); // Terminate OK
    } else {
        // Father process
        printf("[%s] Father process (PID=%d)\n", timestamp(), getpid());
        wait(&status); // Wait for child to finish
    }
    return 0; // Return OK
}
```

Se pide **analizar, compilar y ejecutar el código 'pexample1.c'** para poder **contestar a las siguientes preguntas**.

Preguntas para contestar en el informe de prácticas. Responde a cada pregunta en el informe intentando explicar el porqué de tus respuestas.

- ¿Comparten el padre y el hijo el mismo código del programa? ¿Ejecutan el padre y el hijo las mismas líneas del código? ¿Cómo discriminamos qué partes del código ejecuta cada proceso?
- ¿Qué función realiza la función getpid()? ¿Puede getpid() devolver el mismo resultado a dos procesos distintos? ¿Puede devolver cero? Describe qué pruebas has realizado para comprobarlo.
- ¿Qué sucedería si el hijo llamara a wait()?
- ¿Es cierto que, dado que el padre crea al hijo con fork(), el padre siempre se ejecuta primero? ¿Cómo lo puedes comprobar?

Código para desarrollar y presentar en el informe con los comentarios necesarios

Extender el código anterior para crear 50 procesos hijos. Cada proceso hijo tiene que imprimir su timestamp y terminar. El proceso padre deberá esperar la finalización de todos los procesos hijos y, solo entonces, terminar.

- ¿Qué se observa en los resultados al ejecutar múltiples veces el programa?
- ¿Genera siempre los mismos resultados?
- ¿Se ejecutan los procesos siempre en el mismo orden? ¿Cómo lo has comprobado? ¿Por qué pasa esto?

Ejercicio 2: Espacio de memoria de procesos y concurrencia

En el siguiente ejemplo ('pexample2.c') se muestra como ambos procesos padre e hijo imprimen por pantalla el valor de la variable 'number'. Se pide **analizar, compilar y ejecutar el código 'pexample2.c'** para poder **contestar las siguientes preguntas**.

```
int main(int argc, char** argv) {
    int pid, status, number = 300;
    // Create a process (fork)
    pid=fork();
    if(pid==0) {
        // Child process
        number = 400;
        printf("Child process: Number is %d\n", number);
        // Terminate OK
        exit(0);
    } else {
        // Father process
        number = 500;
        printf("Father process: Number is %d\n", number);
        // Wait for child to finish
        wait(&status);
    }
    return 0; // Terminate OK
}
```

Cuestiones para resolver y describir el resultado obtenido y su justificación en la memoria final.

- ¿Imprimen los dos procesos el mismo número por pantalla? ¿Por qué?
- ¿Qué sucedería si ambas llamadas a printf() se realizaran antes de la asignación a 'number'? ¿Por qué?
- ¿Depende el resultado de la ejecución del programa del orden en que los dos procesos asignan valor a la variable 'number'? ¿Por qué?
- ¿En qué punto del programa se reserva espacio para cada variable 'number'?
- ¿Es necesario más de una CPU para ejecutar el código anterior? En caso negativo, describe cómo se ejecutarían los dos procesos en un procesador con 1 sola CPU. Por ejemplo, puedes explicar las instrucciones que se van ejecutando de cada proceso.

Ejercicio 3: Comunicación entre procesos. Pipes.

Podemos ver como la pregunta natural que se nos plantea después de analizar el ejemplo anterior es cómo podemos comunicar datos entre dos procesos. Para ello analizaremos el ejemplo 'pexample3.c'. Este programa crea un pipe (unnamed pipe) y posteriormente llama a `fork()` creando un proceso hijo. De esta forma, ambos procesos tienen acceso a los extremos de escritura y lectura del pipe. En el ejemplo, el padre escribe en el pipe un dato de tipo entero, mientras que el hijo espera leyendo del otro extremo del pipe a que el padre le envíe el dato. Puedes revisar la teoría sobre comunicación de procesos usando pipes en los puntos 6.6.4 y 6.6.10 del libro Sistemas Operativos de Pedro de Miguel

Se pide **analizar, compilar y ejecutar el ejemplo para poder contestar las siguientes preguntas.**

```
int main(int argc, char **argv) {
    int fd[2];
    int pid, status;
    // Create an unnamed pipe (store pipe descriptors into fd)
    pipe(fd);
    // Fork
    if ((pid=fork())==0) {
        // Child process
        printf("Child process: Created\n");
        // Read integer from pipe
        int number = 0;
        read(fd[0], &number, sizeof(int));
        // Print number
        printf("Child process: Number read %d\n", number);
        // Terminate OK
        exit(0);
    } else {
        // Father process
        int number = 900;
        // Write integer into the pipe
        printf("Father process\n");
        write(fd[1], &number, sizeof(int));
        printf("Father process: Number written\n");
        // Wait for child to finish
        wait(&status);
    }
    return 0; // Terminate OK
}
```

Preguntas:

- ¿Qué sucede si el padre envía el dato antes que el hijo este leyendo desde el otro extremo? ¿Se pierde el dato? ¿Salta un error?
- ¿Qué sucede si el hijo quiere leer el dato antes que el padre haya escrito en el otro extremo? ¿El hijo ignorará la llamada a `read()`?
- ¿Qué le sucederá al padre si la pipe está llena? ¿Qué le sucederá al hijo si la pipe está vacía?

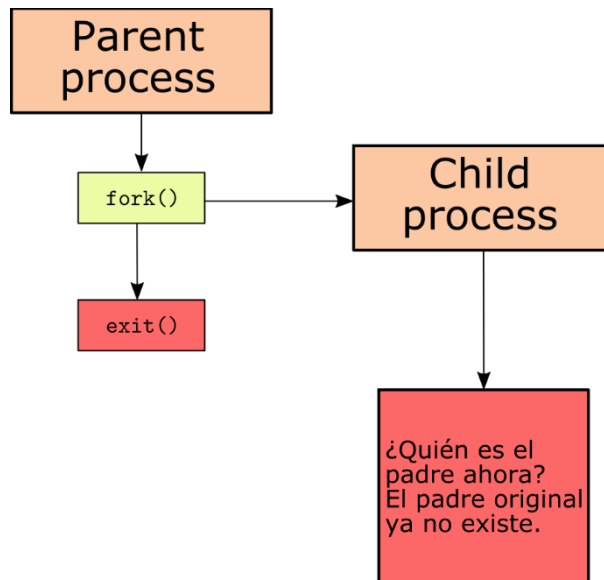
- ¿Qué sucede si intentamos leer de una pipe cuyo otro extremo se ha cerrado? ¿Qué sucede si intentamos escribir en una pipe cuyo otro extremo se ha cerrado?

Extender el código anterior para que el proceso padre le envíe al proceso hijo un buffer de caracteres (string) con el contenido “SOY TU PADRE”.

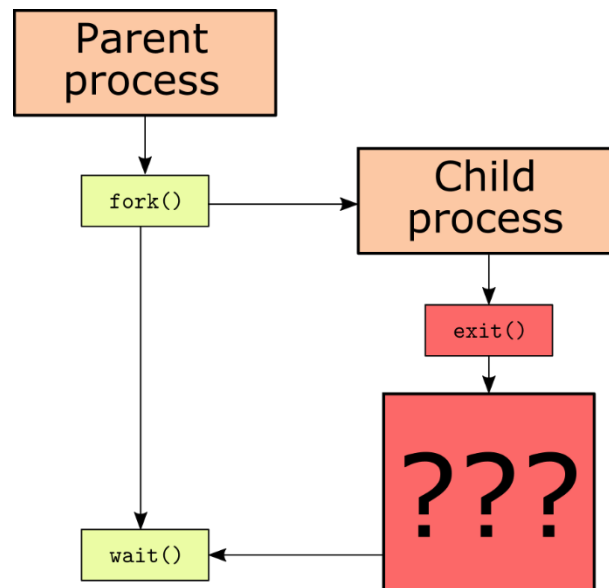
Ejercicio 4: Ciclo de vida de los procesos

El tiempo de vida de un proceso padre y un proceso hijo normalmente no es la misma, o termina antes el padre, o termina antes el hijo. Esto genera varios problemas con los que el *kernel* tiene que lidiar:

1. Si el padre termina antes que el hijo: ¿Quién es el proceso padre, una vez ha *muerto* el proceso padre original? Los procesos que se quedan sin padre se denominan *orphan processes*.



2. Si el hijo termina antes que el padre: ¿Qué pasa con el proceso hijo entre que este termina y el padre hace el `wait()`? Estos se convierten en *zombie processes*. No se pueden matar (ni con la señal SIGKILL) hasta que el proceso padre haga el `wait()`.



Se tiene el código `orphan.c` que deja el proceso hijo huérfano. Hacer que el proceso hijo escriba por pantalla el PID de su nuevo padre. ¿Cuál es el PID del nuevo padre? ¿Qué tiene de especial el nuevo proceso padre?

Se tiene el código `zombie.c`, en el cual el proceso hijo termina antes de que el padre haga el `wait()`. Se pide imprimir por pantalla (usando la función `print_child_status()`) el estado del proceso hijo una vez ha terminado ¿En qué estado se encuentra? Seguidamente, enviar una señal `SIGKILL` al proceso hijo ¿Cambia el estado del proceso? ¿Por qué? Volver a revisar el estado después de hacer el `wait` ¿Aún existe el proceso zombie? ¿Por qué?

Ejercicio5: ejecución de comandos con `execvp()`

En este apartado usaremos la familia de funciones `exec()` para crear un pequeño emulador de terminal. La familia de funciones `exec()` reemplazan el ejecutable que está corriendo el proceso por otro. La lógica que tendrá nuestra pequeña terminal será la siguiente:

1. Leemos una línea que nos da el usuario, que será un comando (`ls`, `cat`, `echo`, `ps`...).
2. Nuestra terminal creará un proceso hijo.
3. “Convertiremos” el proceso hijo en el programa que el usuario quiere ejecutar, a través de la función `execvp()` (<https://linux.die.net/man/3/execvp>).
4. La terminal esperará a que el proceso hijo termine de ejecutarse.
5. Volvemos al punto 1.

El esqueleto del código está en el fichero `mini-shell.c`, donde solamente hay que rellenar la función `execute_command()`.

Podeis comprobar el correcto funcionamiento del programa usando comandos básicos de la terminal de Linux como `ls`, `cat`, `echo`, `ps`... Estos deben ejecutarse correctamente.